

Прак
тика

1:

Контекст
до кода

Марат Зимнуров



01

Руководитель группы разработки в AvitoTech, преподаватель искусственного интеллекта в ИТМО и ассистент кафедры ИТиЦЭ ИГХТУ.

02

За 10 лет прошел путь от full-stack-разработчика до руководителя команд, но так и не перестал писать код, разбирать архитектуру и проводить инженерные эксперименты.

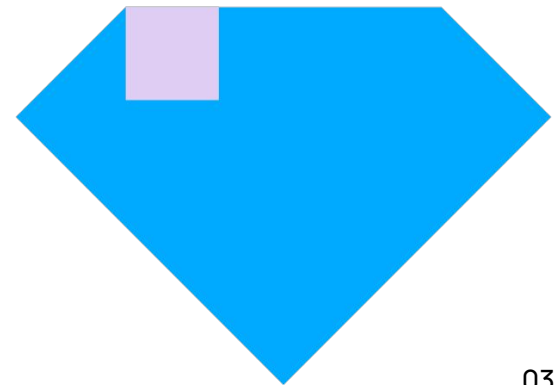
03

Автор научных работ, образовательных и open-source-проектов, спикер HolyJS и CodeFest. Исследую функциональное программирование, LLM и формальные способы объяснить машине, что правдоподобный код — еще не обязательно рабочий.

04

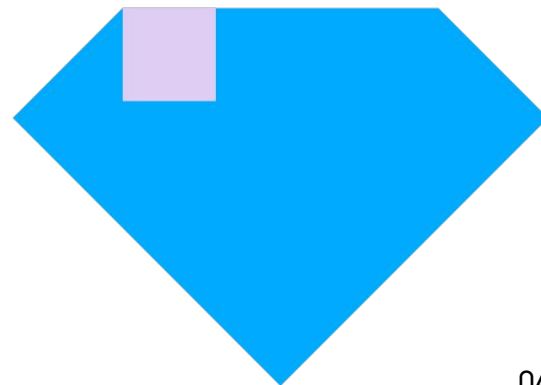
Убежден, что все вокруг — функция. Теперь пытаюсь доказать это формально.

Цель — освоить базовый промптинг на документах будущего проекта и сравнить короткий запрос с master prompt



FROM → TO

идея + короткий запрос
→ документы + master prompt
+ проверка



90 минут

0–5

Цель и учебный кейс

5–20

Форк + OpenCode + токен

20–32

Стили промптинга

32–42

Запуск 1: zero-shot

42–52

Разбор пробелов

52–65

Context Pack + master prompt

65–75

Собрать версию 1

75–85

Запуск 2: master prompt

85–90

Сравнение

В вашем
форке
уже есть
TRAINING_
PR.diff

Настройте OpenCode

1. Fork → Clone → cd ITMOv2 и **создайте отдельную ветку для практики**

2. Проверьте: `opencode -version`

3. `opencode` → `/connect` → VseLLM

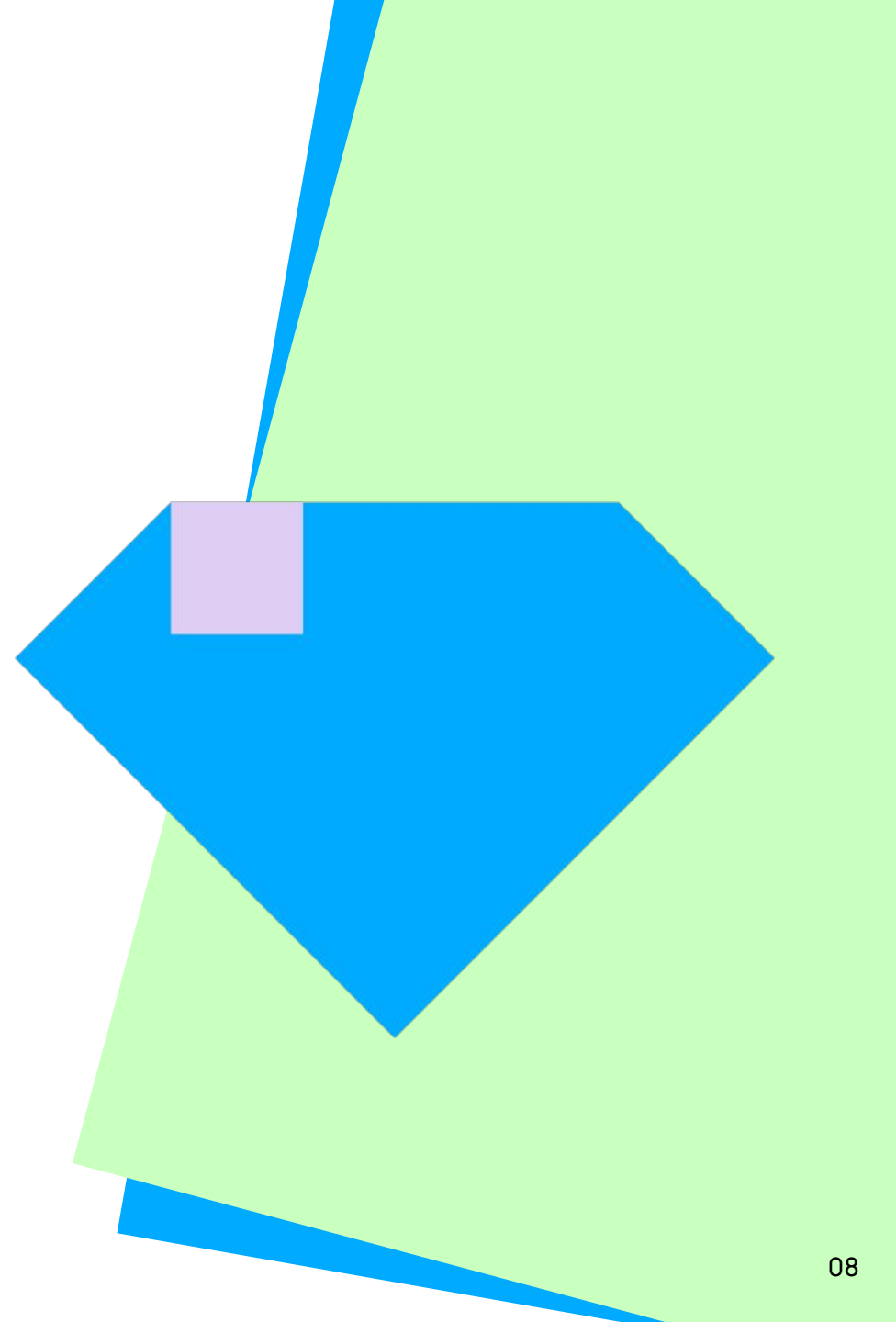
4. Вставьте выданный токен

5. `/models` → модель преподавателя

6. `opencode run` «Ответь: READY»

От запроса к структуре

1. Zero-shot — без примеров: baseline
2. Few-shot — примеры результата
3. RCTF — Role, Context, Task, Format
4. Tree of Thoughts — варианты + критерии



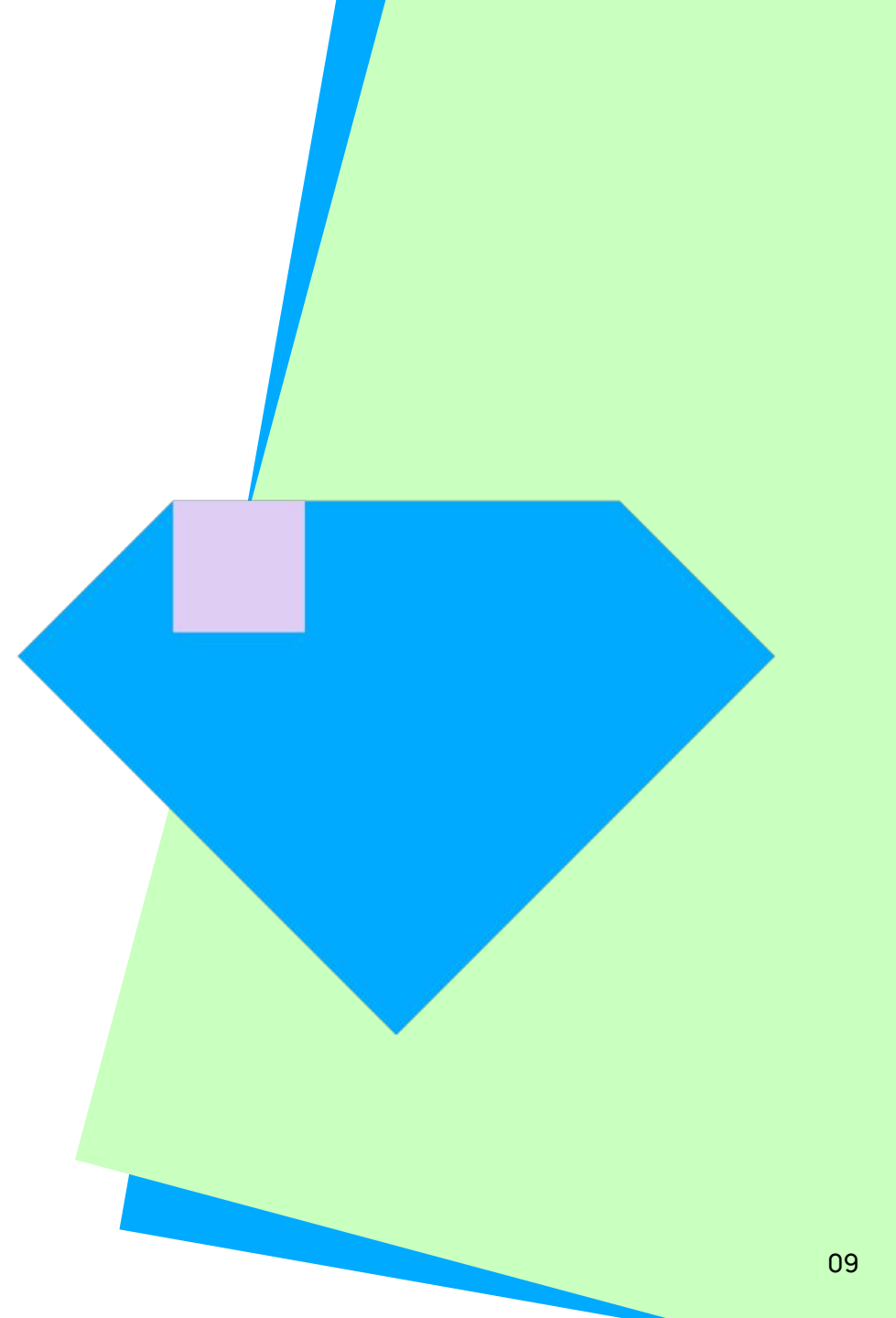
От структуры к агенту

1. Chain of Verification – claim → evidence → check

2. ReAct – action → observation → stop

3. Chain of Thought – внутри модели; не сдаём

4. Master prompt – полный контракт агента



Zero-shot и few-shot на одном diff

Zero-shot

«Используй только приложенный diff.
Найди до 3 проблем.
Для каждой укажи file:line, evidence
и проверку.»

Few-shot

«Следи примерам.
Хорошо: app/review_service.py:16
— нарушено SEC-1.
Проверка: diff с token.
Плохо: “Код можно улучшить”.
Верни до 3 замечаний в формате
хорошего примера.»

RCTF и Tree of Thoughts на одном diff

RCTF

«**Role:** reviewer. **Context:** diff + SEC-1, API-1, REL-1.

Task: найди до 3 доказанных рисков.

Format: summary

→ risks{file,line,evidence,risk} → checks.»

Tree of Thoughts

«Собери 3 стратегии: security-first, reliability-first, balanced.

Сравни по evidence и покрытию правил.

Выбери одну и верни только итоговое ревью.»

Verification и ReAct на одном diff

Chain of Verification

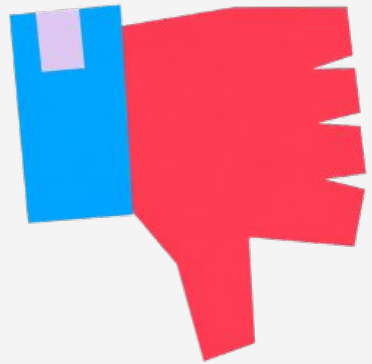
- «1. Сформируй кандидаты в риски.
- 2. Для каждого найди строку diff или правило.
- 3. Удали всё без evidence.
- 4. Верни проверенный список.»

ReAct

«Цикл: открыть фрагмент diff → сверить правило →
добавить замечание или пропустить.
Стоп после 3 рисков.
Не меняй файлы и не запускай shell.»

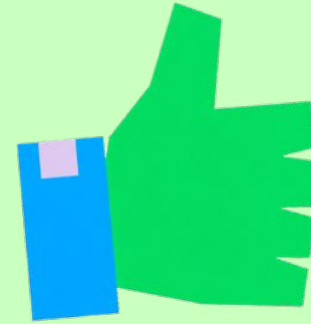
Chain of Thought

Нужен проверяемый результат, а не внутренний монолог.



Не так

«Покажи все свои мысли шаг за шагом.»

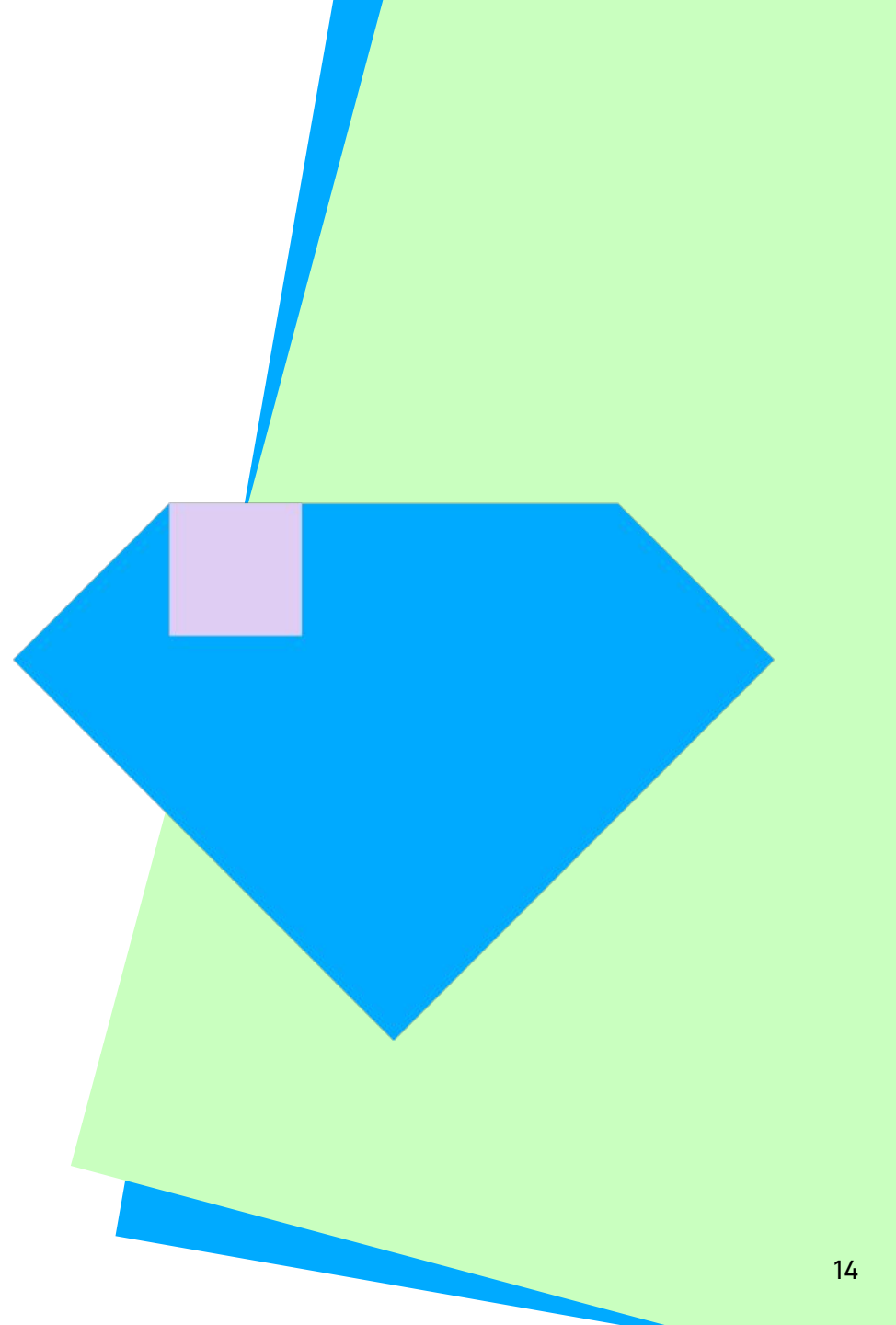


Так

«Не выводите скрытые рассуждения.
Верни:
1. вывод; 2. evidence: file:line
или правило; 3. альтернативу;
4. уверенность; 5. проверку.»

Запуск 1: zero-shot

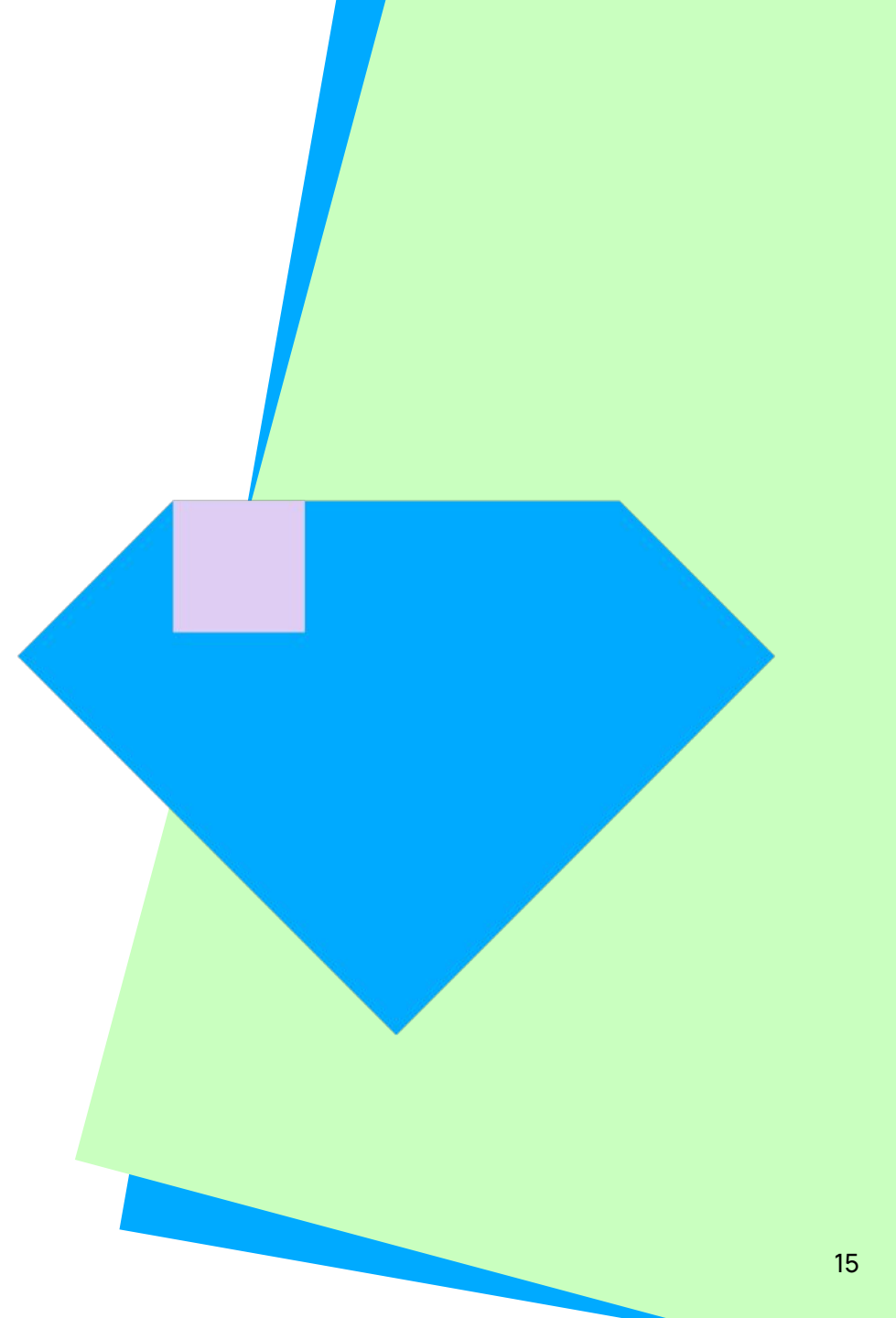
1. Откройте новый чат
2. Приложите только TRAINING_PR.diff
3. Напишите: «Посмотри PR и найди проблемы»
4. Сохраните ответ как P1-01 в prompts.md
5. Не давайте CASE.md, репозиторий и web



После запуска 1

1. Что модель выдумала?
2. Есть ли ошибки?
3. Какого правила или формата не хватило?
4. Какое действие AI нужно запретить?

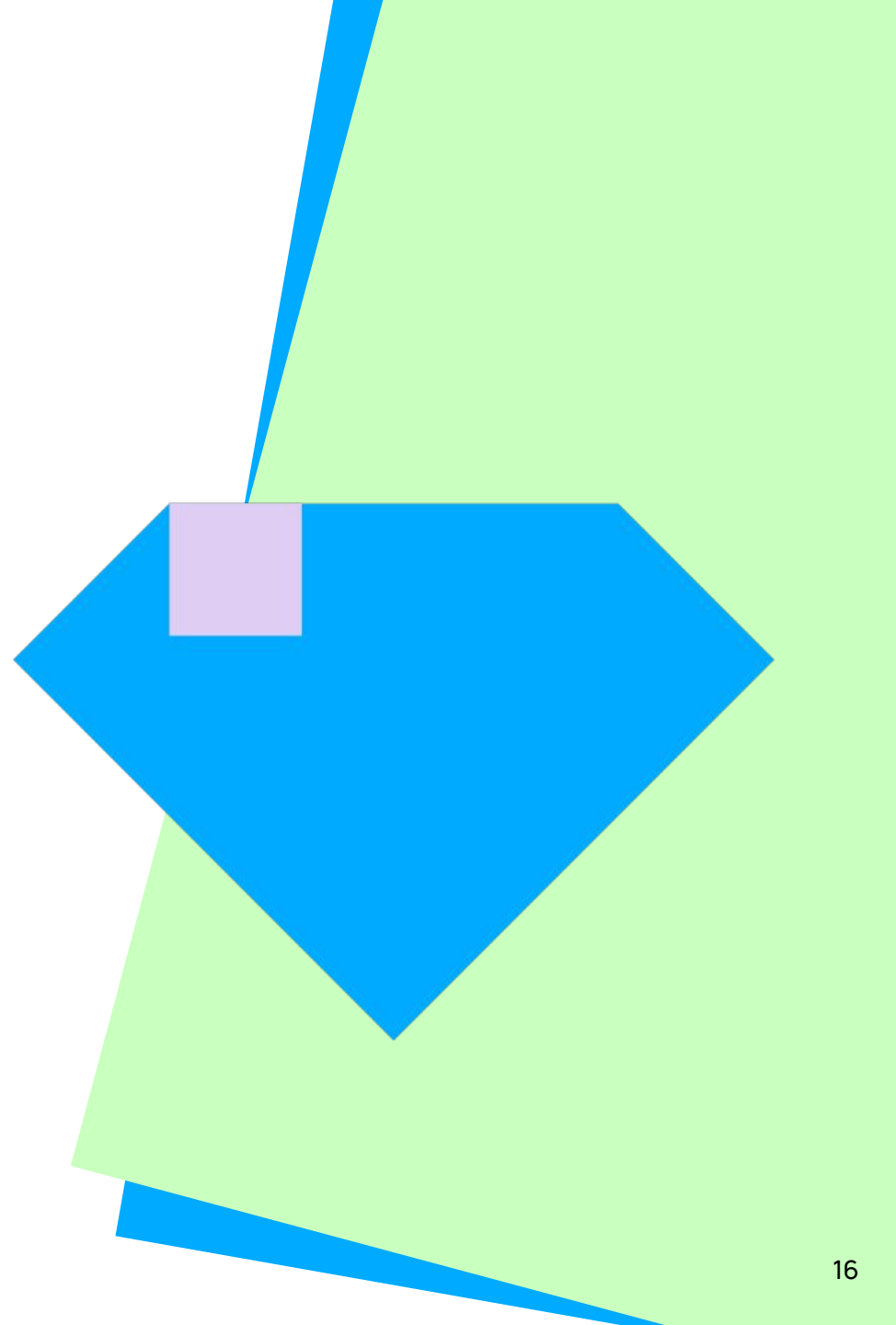
Не улучшайте запрос: это baseline



На паре — формализируем вывод агента

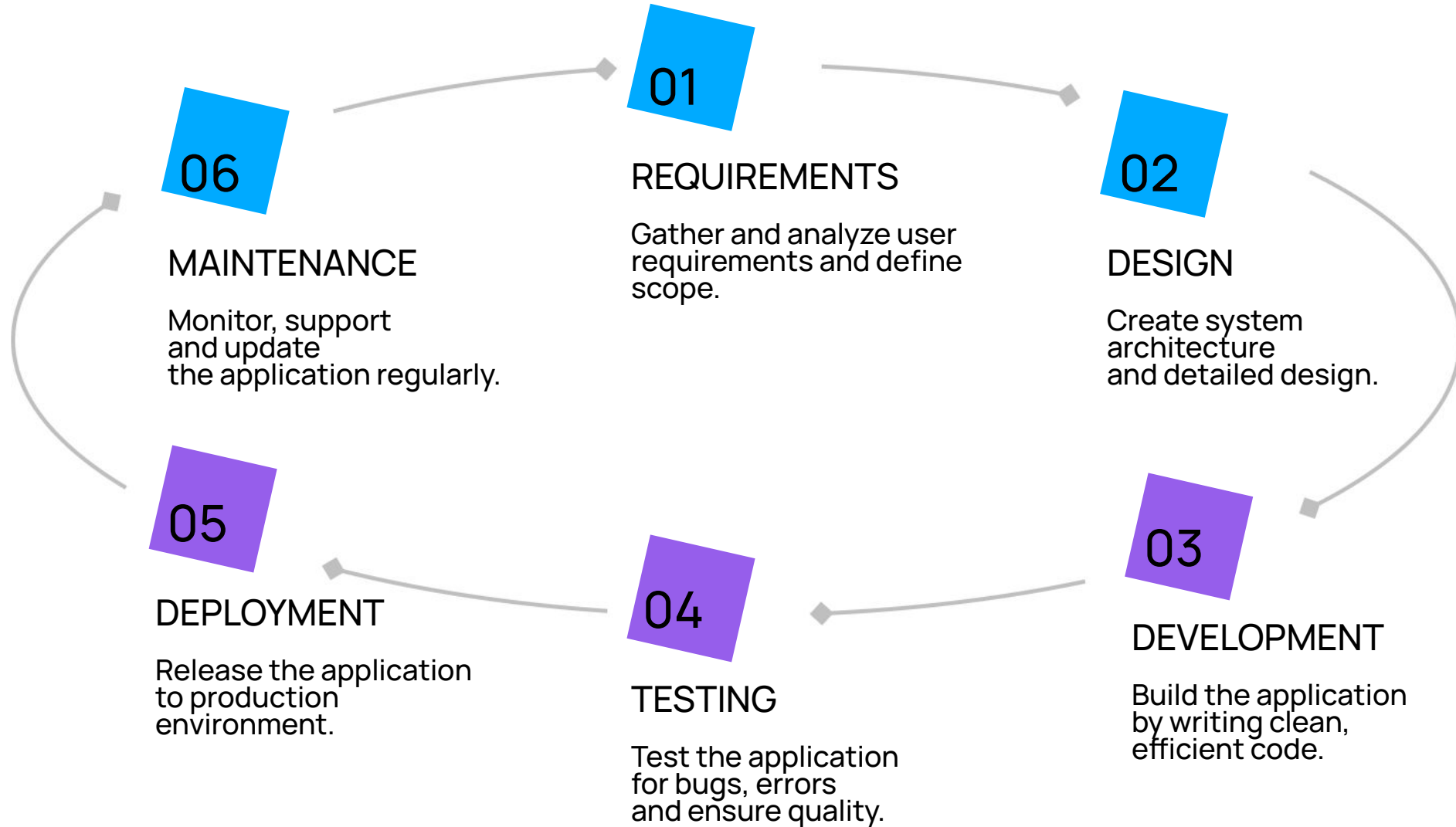
1. context.md: факты, источники, ограничения
2. problem.md: пользователь, проблема, метрика
3. prompts.md: P1-01 и пробелы контекста
4. prompts.md: каркас Master Prompt v1

Остальные файлы — домашняя работа



Разложите один сценарий

SDLC – Software Development Life Cycle



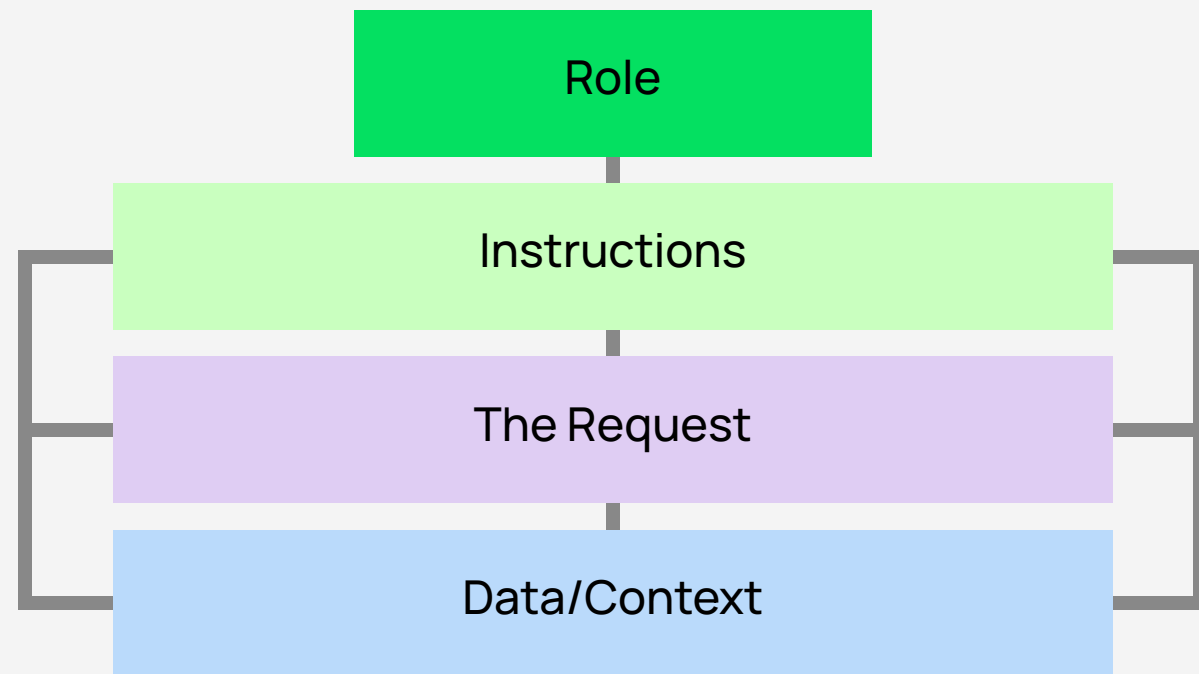
Master prompt

Контракт работы AI-агента

Context Pack

Выбранные для одной задачи факты, правила, примеры и ограничения. Это часть master prompt, а не конечный результат.

Prompt Structure



Структура master prompt

01

Goal + Role

02

Inputs + Sources
+ Context Pack

03

Task +
Deliverables

04

Output Format

05

Allowed /
Forbidden + Stop

06

Workflow +
Verification + DoD

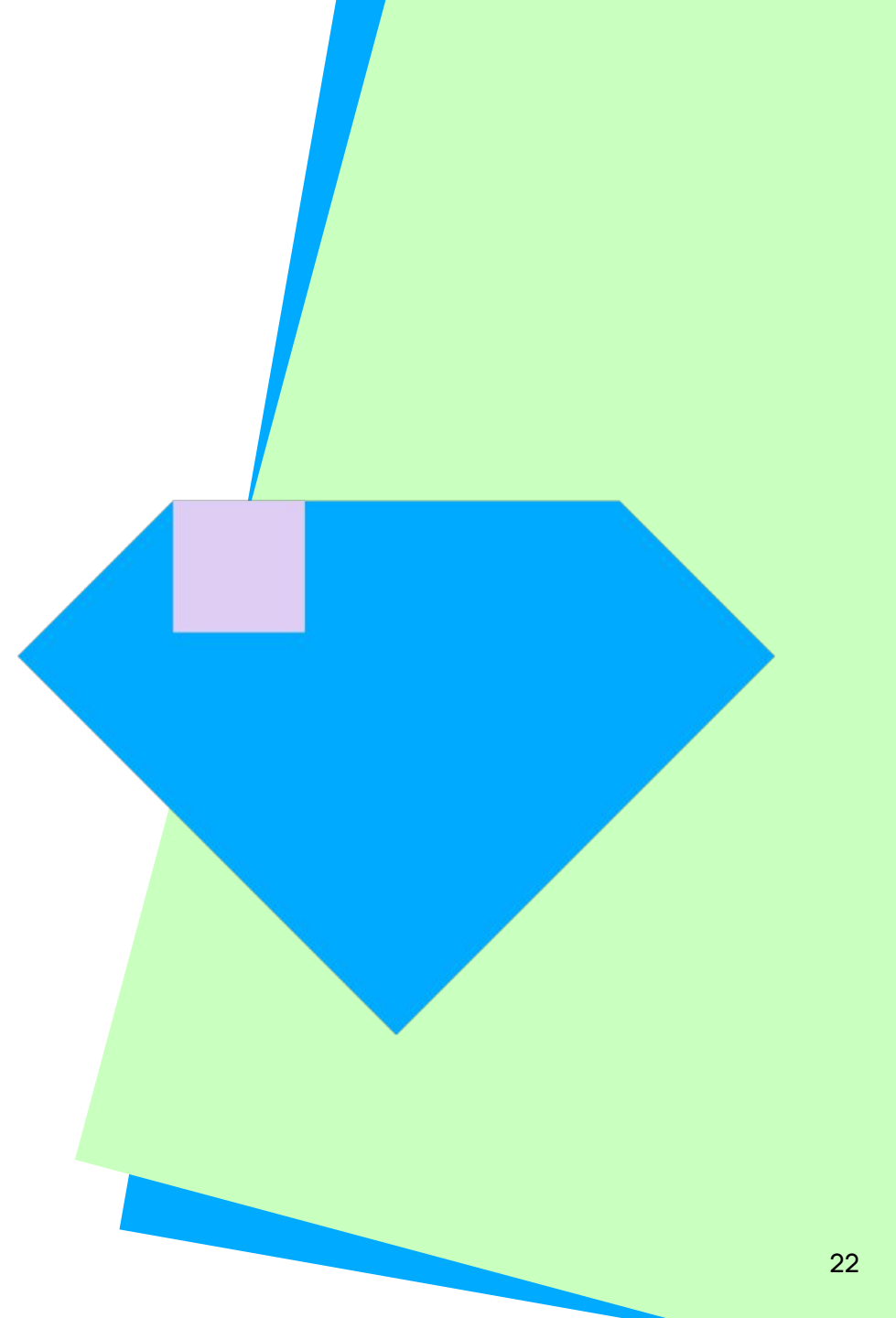
Master Prompt v1

В коротком виде

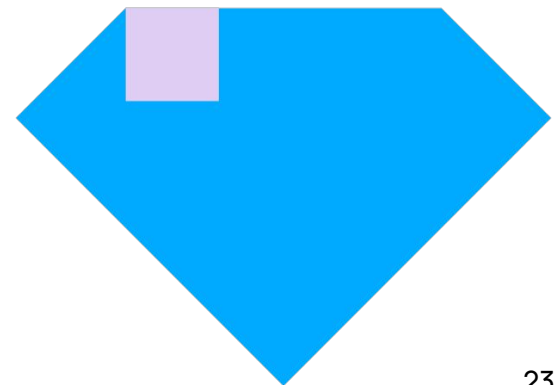
«**Role:** AI-reviewer.
Inputs: diff + SEC-1...QA-1.
Return: summary + ≤ 3 risks + checks.
Risk: file:line + evidence + rule.
Forbidden: approve, merge, edit.
Не придумывай правила.
Flow: candidate → evidence → check.
Нет evidence → пропусти.
Done: evidence + check для каждого риска.»

Запуск 2: master prompt

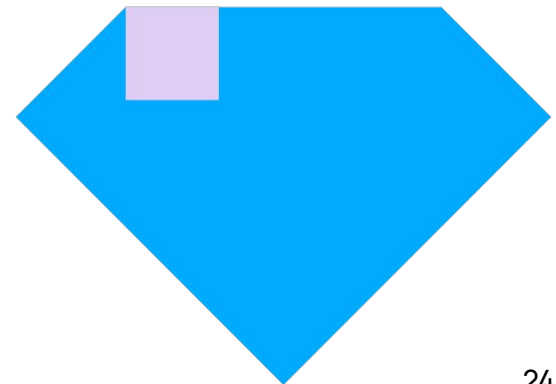
1. Тот же TRAINING_PR.diff
2. Та же модель и задача заполнить context.md, problem.md и prompts.md
3. Добавьте Master Prompt v1
4. Не подсказывайте модели
5. Проверьте что модель заполнила все файлы



Сравните первый запуск
и второй запуск.
Что изменилось в выводе и
размышлениях агента?



После пары откройте
PR внутри своего
репозитория, где получите ОС
от ревьювера



Домашняя работа

1. Доведите до конца 11 обязательных файлов
2. Положите Master Prompt v1 в prompts.md
3. Запустите make test и открыть PR
4. Получите и обработайте замечания ревьюера

Что вы сдаёте

Заполняются несколько файлов, а не один общий ответ:

Файл	Что в нём должно быть
context.md	AS IS: продуктовый и командный контекст, источники и ограничения
problem.md	пользователь, проблема и измеримые продуктовые метрики
analysis.md	AS IS и TO BE процесса в выбранной нотации
product_management.md	use cases, user stories и сценарии
project_management.md	инкременты, роли людей/AI и план поставки
adr.md	архитектурное решение и Mermaid-схема
tests_unit.md	unit-проверки
tests_integration.md	integration-проверки
tests_load.md	нагрузочные проверки и ожидаемые пределы
tests_e2e.md	сквозные сценарии
prompts.md	запросы, результаты, ссылки на изменённые артефакты и собственная оценка

В каждом содержательном файле есть короткий раздел «Как использовали AI». Он нужен, чтобы ревьюер увидел назначение запроса, тип промпта, ссылку на строку в `prompts.md` и вашу собственную проверку результата.

Критерии

- 10 баллов

Структура и Make-команды

1 балл

Контекст, проблема, метрики

2 балла

TO BE, use cases и план

2 балла

ADR и архитектура

2 балла

4 уровня тестов

2 балла

master prompt
+ 2 запуска

1 балл

Готово, если:

Другой инженер открыл ваш PR
Понял задачу и границы агента
Повторил запуск
И проверил результат